

Arithmetic Logic Unit (ALU)

There is a great deal of technical detail in the design of arithmetic logic units. At this point, so as not to get stuck in the weeds, we will postpone the complexity to a later time.

For our purposes, we will assume the ALU is a **combinational circuit** that **simultaneously** performs **all possible** arithmetic and logical operations, then **selects** one of the results using a **multiplexer**.

ALU (continued)



ALU (continued)

This assumes you have w bits to choose the operation, therefore (therefore 2^w choices). A simple example might be:

Operation	Bits	Output
NOP	0	C to high-impedance
ADD	1	$C=A+B$
SUB	2	$C=A-B$
AND	3	$C=A \text{ AND } B$
OR	4	$C=A \text{ OR } B$
...	...	...
SHIFT L	2^w-1	$C=A \ll B$

ALU (continued)

Condition codes (as you have seen before) remember something useful about the previous operation:

Symbol	Name	Meaning
Z	Zero	The last result was 0
N	Negative	The sign bit was set
C	Carry-out	There was a carry out from the last operation
V	Overflow	The last operation caused arithmetic overflow
...	...	...

This could be as simple as a flip-flop in the ALU.

Registers

In addition to the ALU, microprocessors contain **registers**.

Registers, as we have already seen, are a set of flip-flops that are controlled together (in parallel). We can first broadly classify registers as to whether they are **user-visible** or **internal**. User-visible registers can be further subdivided as **special purpose** vs **general purpose**.

Internal Registers

Almost all microprocessors have the following, which exist in the CPU, but the user can't directly access.

Symbol	Name	Purpose
PC	Program counter	Address of the next instruction to execute
IR	Instruction register	Instruction currently under execution
CC	Condition codes	ALU condition codes and CPU operating modes
MAR	Memory address reg	Address to be used for the next memory operation
MDR	Memory data reg	Data most recently fetched from memory

User-visible registers

Depending on processor style, you will find:

Name	Purpose
General purpose registers	Can be any operand or destination for any instruction. This is common in modern processors
Data registers	Can hold data only. In some cases, can be only one kind of data (accumulator, counter, etc)
Address registers	Can hold addresses only
Stack pointer	Points at the execution stack. Might just be a special case of address register. Might not exist at all.

Registers (cont)

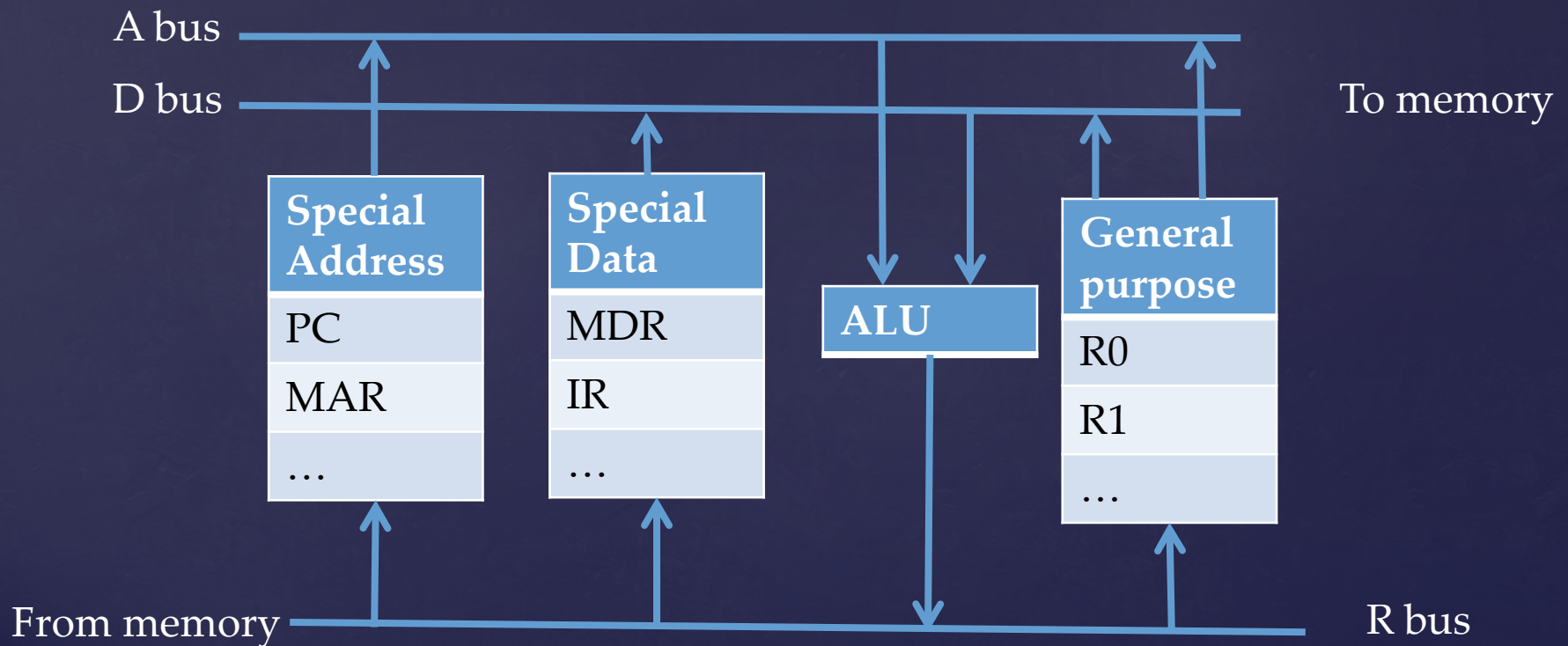
We will see later in the course that some of the performance improvements of modern processors have changed the type, number and function of many common CPU registers.

For our purposes now, we will assume each register is equipped with:

Control input	Meaning
OE_x	Output enable to bus X. Drive the output bus X when high. Tri-state otherwise
LD	Load input. Update the content when high. Ignore input otherwise.

CPU Datapath

With these definitions, this is the datapath of a microprocessor:



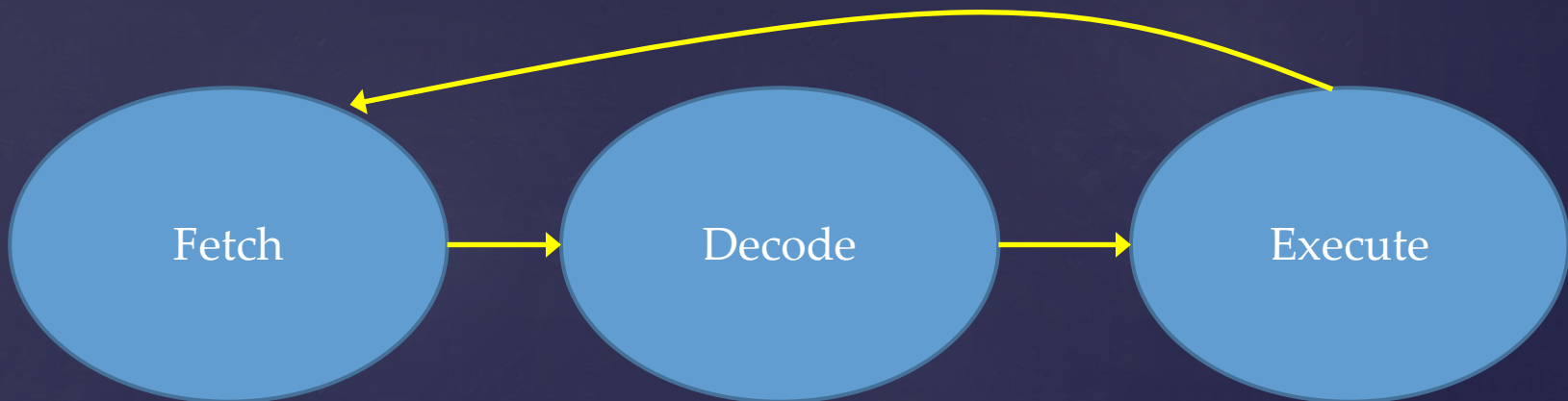
A note on memory access

We will look at options for memory access in detail later. For now we make the following assumptions. First, assume that memory is synchronous, so operations occur in this clock cycle. Second, memory has the following interface:

Signal	Name	Dir	Meaning
A[]	Address	IN	Specifies the memory address for all operations
D[]	Data	IN/OUT	OUTPUT from memory on READ. INPUT to memory on WRITE.
WR	Write	IN	Write D to address A
RD	Read	IN	Drive D with contents of A

CPU operation

We are now ready to talk about what a CPU does. Basically a CPU is this state machine:



Of course it gets more complicated than that, but that's basically all there is to it. Let's look at what this means in more detail.

Fetch

The operation we call Fetch means the following:

1. Get the word from memory at the address currently in PC and load it into IR.
2. Increment PC so it points at the next instruction in memory

Without making further assumptions about the PC, this is a two-cycle operation:

Of course, it's called the program **counter** for a reason. If we assume it is a counter, with a PC_CE (count enable) input, we need only one cycle.

Cycle 1:
PC_OEA=1
IR_LD=1
RD=1

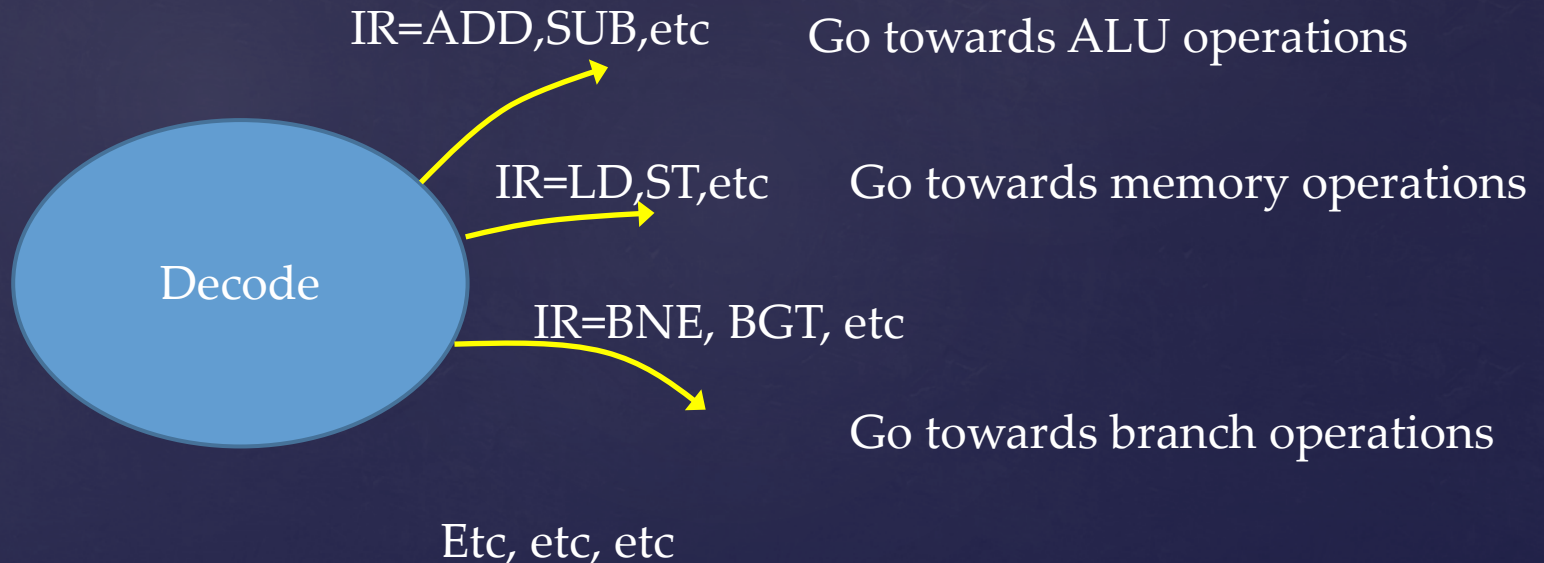
Cycle 2:
PC_OEA=1
ALU_OP=INC
PC_LD=1

Cycle 1:
PC_OEA=1
IR_LD=1
PC_CE=1
RD=1

Decode

After **Fetch** the IR contains the **opcode** of the next instruction to execute.

So, the job of Decode (in a Moore machine) is just to decide which branch to follow:



Execute

Of course the path followed for execution depends on the instruction, the addressing mode, etc. For now, let us consider some possible instructions:

1. ADD R1,R2,R3; $R3 \leftarrow R1 + R2$
2. LD R1,#imm; $R1 \leftarrow$ immediate value from instruction
3. BEQ #offs; Branch to $PC + offs$ if $Z = 1$
4. LD R1,R2,R3; $R3 \leftarrow M[R1 + R2]$

ADD R1,R2,R3

This instruction is going to require only a single clock cycle. We need to tell the register file to drive BOTH A and D buses AND load from the Result bus. Something like this:

Cycle 1

R_OEA []=2

R_OED []=1

R_LD []=3

ALU=ADD

LD R1,#imm

We need to load R1 with the next word in the program. Fortunately, that's already where the PC is pointing:

Cycle 1:

PC_OEA=1

R_LD []=1

PC_C=1

BEQ #offs

This one is probably going to need some extra cycles. In the first cycle, we check Z. If Z=0, we're done, but we might as well load the immediate value in either case. In the second cycle, we change PC.

Cycle 1:

if Z=0 goto Fetch

if Z=1 goto Cycle 2

PC_OEA=1

LD_MDR=1

PC_C=1

Cycle 2:

PC_OEA=1

MDR_OED=1

PC_LD=1

ALU=ADD

LD R1,R2,R3

This one almost certainly needs more than one cycle:

Cycle 1:

R_OEA[]=1

R_OED[]=2

ALU=ADD

LD_MAR=1

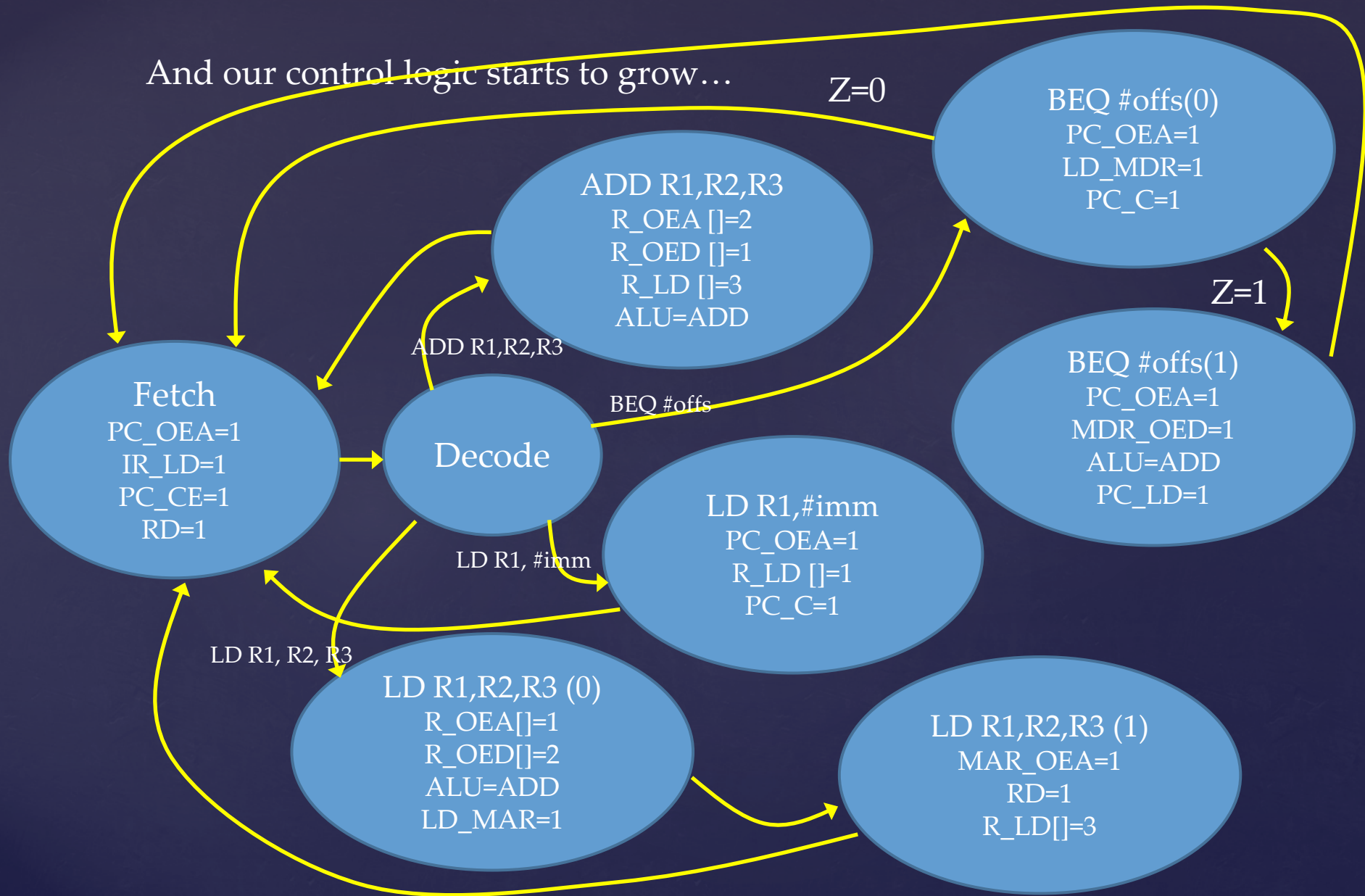
Cycle 2:

MAR_OEA=1

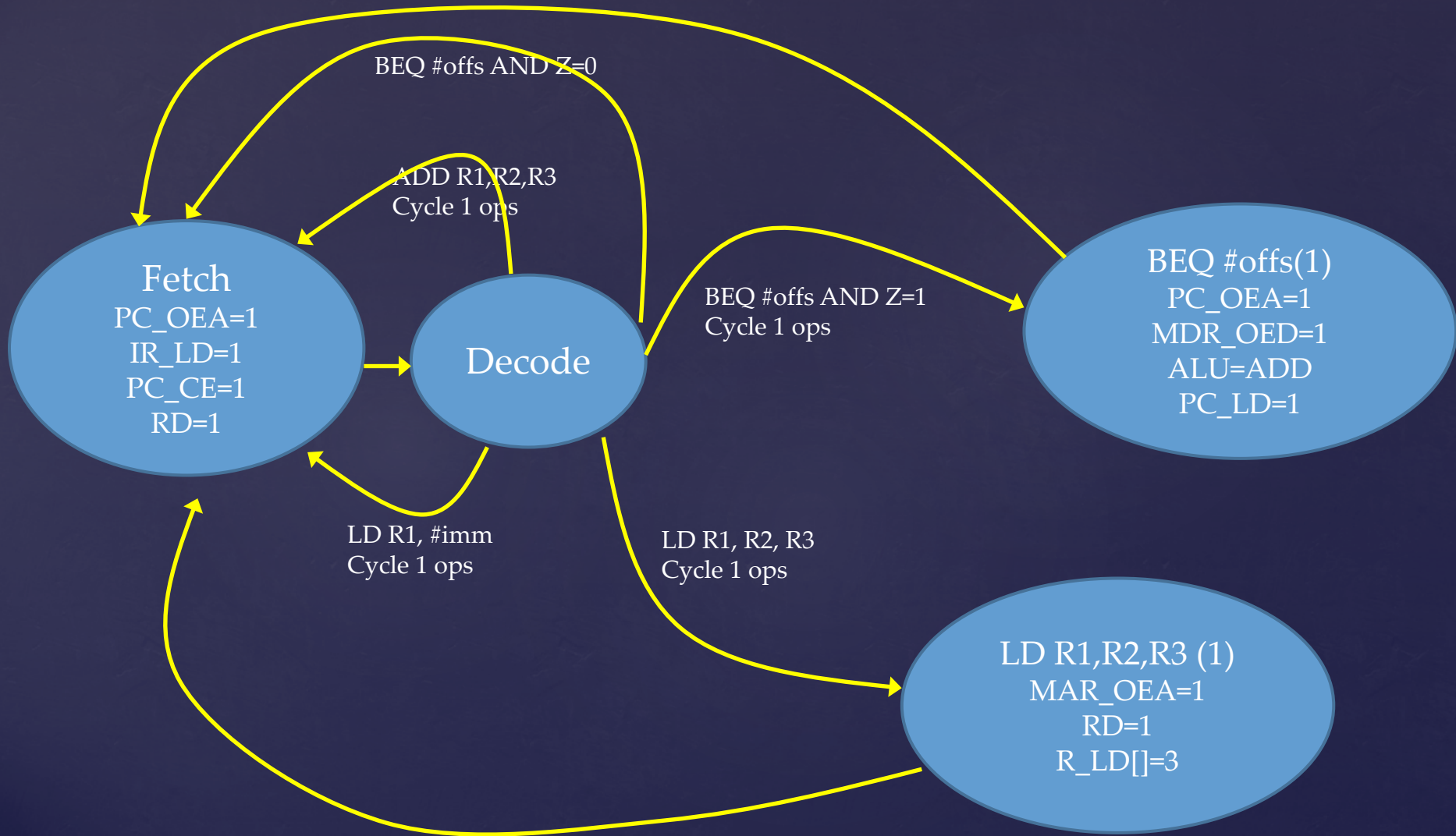
RD=1

R_LD[]=3

And our control logic starts to grow...



If you like Mealy machines...



Hardwired vs Microprogram Control

If we implement the FSM on the previous slide, we will have a control logic for our processor. This is called **hardwired control**. Hardwired control is generally fast and small, but it is not easy to modify after design/layout.

The alternative, as we have seen, is **microprogramming (microcoded) control**.

CPU Microcoding

Let's take a look at the “output” stages from our state machine:

Fetch 1A:

PC_OEA=1
IR_LD=1
RD=1

Fetch 2A:

PC_OEA=1
ALU_OP=INC
PC_LD=1

Fetch 1B:

PC_OEA=1
IR_LD=1
PC_CE=1
RD=1

LD R1,#imm 1:

PC_OEA=1
R_LD []=1
PC_C=1

LD R1,R2,R3 1:

R_OEA[]=1
R_OED[]=2
ALU=ADD
LD_MAR=1

LD R1,R2,R3 2:

MAR_OEA=1
RD=1
R_LD[]=3

CPU Microcoding

Just as we saw with our simpler ASM models, each clock cycle is characterized by an **output word** that looks like this:

PC_ OE	PC_ LD	PC_ C	IR_ LD	ALU[]	MAR_ LD	MAR_ OE	RD	WR	...
-----------	-----------	----------	-----------	-------	------------	------------	----	----	-----

This word will be very long, and not really human readable, but it will contain everything the CPU should do on every clock cycle.

CPU Microcode Sequence

The tricky part is sequencing. As we have seen, microcode must be rearranged carefully to minimize branching. Ideally, blocks of similar microcode will only exist in one place. Then the **instruction set opcodes** can be defined to simplify the process of jumping around. Consider the following, from the old **Motorola HC12**

Instruction	Opcode	Instruction	Opcode
ADDA dir	9B	ADDB dir	DB
ADCA dir	99	ADCB dir	D9
ADDA #imm	8B	ADDB imm	CB
ADCA #imm	89	ADCB #imm	C9

Microcode Structures

Microcodes are defined as:

Horizontal: Keep one bit per control line. This is the kind we have seen. Easy to understand, but usually wasteful since most of the bits are zero most of the time.

Vertical: Many control words are not possible. So, you can define the set of possible control words, then put THEM into a lookup table and store only their ID. The storage required is less, but the complexity is greater.

Nanoprogramming: In the nanoprogramming concept, we analyze the entire program to find the smallest set of unique microinstructions.

Suppose there are A microinstructions of B bits in length. A horizontal microstore would require AB bits of storage. If only n of them are unique, we need $A \log(n)$ in the microstore and nB in the decoder.